

Optimizers in Deep Learning

Gradient Descent → SGD → Momentum → AdaGrad → RMSprop → Adam

A structured study guide on how neural networks learn their parameters — the algorithms that decide *how* weights are updated during training, why plain Gradient Descent struggles, and how modern optimizers fix those problems.

1. What Is an Optimizer?

An **optimizer** is the algorithm used to update a neural network's weights and biases during training so that the **loss function** is minimized. Training a neural network is fundamentally an optimization problem: given a loss surface (a function of all the weights), we want to find the set of weights that gives the lowest possible loss.

Every optimizer answers the same core question: **given the current weights and the gradient of the loss, how exactly should we change the weights?** The general update rule for almost every optimizer builds on this base form:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla L(w_{\text{old}})$$

- w = weight (parameter) being updated
- η (**eta**) = learning rate — controls the step size
- $\nabla L(w)$ = gradient of the loss function with respect to w (direction of steepest increase)

We move *opposite* to the gradient because the gradient points toward increasing loss — moving against it decreases the loss.

2. Role of Optimizers in Training

Optimizers matter because they directly control three things that decide whether training succeeds, fails, or wastes time:

- **Speed of convergence** — how quickly the model reaches a good (low-loss) solution
- **Stability of training** — whether the loss decreases smoothly or oscillates / diverges
- **Quality of the final solution** — whether the model lands in a good minimum or gets stuck in a poor one (e.g. a sharp local minimum or saddle point)

A neural network's loss surface is typically high-dimensional, non-convex, and full of irregular terrain — flat plateaus, steep cliffs, narrow ravines, saddle points, and many local minima. The optimizer's job is to **navigate this terrain efficiently**, much like choosing a smart path down a foggy, uneven mountain rather than just always stepping straight downhill at a fixed pace.

This is why simply picking a good architecture and loss function isn't enough — the choice of optimizer (and its hyperparameters, especially the learning rate) often determines whether a model trains well at all.

3. Types of Optimizers — Overview

Optimizers can be grouped into a rough family tree based on what they improve upon:

Optimizer	Core Idea	Key Benefit
Batch Gradient Descent	Use the entire dataset to compute one gradient per update	Stable, accurate direction
Stochastic GD (SGD)	Use a single random sample per update	Fast updates, can escape shallow minima
Mini-batch GD	Use a small batch (e.g. 32–256 samples) per update	Balances speed and stability
Momentum	Accumulate a moving average of past gradients	Smooths out oscillations, speeds up in consistent directions
Nesterov Accelerated GD (NAG)	Look ahead before computing the gradient	Corrects momentum's overshoot
AdaGrad	Adapts learning rate per-parameter using sum of squared past gradients	Good for sparse features
RMSprop	Like AdaGrad, but uses a decaying average instead of a sum	Prevents the learning rate from shrinking to zero
Adam	Combines Momentum (1st moment) + RMSprop (2nd moment)	Fast, stable, the default choice for most deep learning tasks

3.1 Gradient Descent and Its Variants

Batch Gradient Descent computes the gradient using the *entire* training set before making one update. This gives a very accurate gradient direction but is computationally expensive and slow for large datasets.

Stochastic Gradient Descent (SGD) updates weights using just *one* training example at a time. This makes updates very fast and introduces useful noise that can help escape shallow local minima — but the path to the minimum becomes noisy and erratic.

Mini-batch Gradient Descent is the practical middle ground used almost everywhere in real deep learning: it computes the gradient over a small batch of samples (commonly 32, 64, 128, or 256), giving a good trade-off between the stability of batch GD and the speed of SGD. This is what's typically used by default — and many discussions of "SGD" in deep learning frameworks actually refer to this mini-batch version.

3.2 Momentum-Based Methods

Momentum adds a fraction of the previous update to the current one, similar to a ball rolling downhill that builds speed. This smooths the path, reduces oscillation in directions where the gradient flips sign frequently, and accelerates movement in directions where the gradient is consistently pointing the same way.

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w) \rightarrow w = w - \eta \cdot v_t$$

Nesterov Accelerated Gradient (NAG) improves on Momentum by first taking a step in the direction of the accumulated momentum, then computing the gradient at that *look-ahead* point rather than at the current position. This gives the optimizer a small correction before it overshoots, leading to faster and more stable convergence.

3.3 Adaptive Learning Rate Methods

Instead of using one global learning rate for every parameter, adaptive methods give **each parameter its own effective learning rate**, based on how that parameter's gradients have behaved historically.

AdaGrad divides the learning rate of each parameter by the square root of the sum of all its past squared gradients. Parameters that have received large gradients repeatedly get their learning rate shrunk; parameters with small/infrequent gradients keep a relatively larger learning rate. This is great for sparse data (e.g. NLP with rare words) but has a major flaw: the accumulated sum only grows, so the effective learning rate keeps shrinking and can eventually become so small that learning effectively stops.

RMSprop (Root Mean Square Propagation) fixes AdaGrad's biggest weakness. Instead of accumulating *all* past squared gradients forever, it keeps an **exponentially decaying moving average** of squared gradients — so older gradients matter less over time. This keeps the effective learning rate from shrinking to zero, making it suitable for non-convex, non-stationary problems like deep neural network training.

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1 - \beta) \cdot g_t^2 \rightarrow w = w - (\eta / \sqrt{E[g^2]_t + \epsilon}) \cdot g_t$$

3.4 Adam — The Default Choice

Adam (Adaptive Moment Estimation) combines the best of both worlds:

- **First moment (mean of gradients)** — like Momentum, it smooths the gradient direction
- **Second moment (mean of squared gradients)** — like RMSprop, it adapts the learning rate per parameter
- **Bias correction** — both moment estimates are corrected for their bias toward zero during the early steps of training

Because it combines directional smoothing with adaptive step sizes, Adam tends to converge quickly and reliably across a very wide range of problems with minimal tuning — which is why it's the default optimizer in most deep learning frameworks (commonly used with learning rate ≈ 0.001 , $\beta_1 = 0.9$, $\beta_2 = 0.999$).

4. Challenges That Optimizers Must Solve

Plain Gradient Descent struggles on real loss surfaces because of several recurring problems. Understanding these explains *why* each more advanced optimizer was invented.

Challenge	What Happens	Addressed By
Choosing the learning rate	Too high → diverges / overshoots. Too low → painfully slow training.	Adaptive methods (AdaGrad, RMSprop, Adam)
Same LR for all parameters	Some weights need big steps, others need tiny steps — one global rate can't suit all.	AdaGrad, RMSprop, Adam (per-parameter rates)
Local minima & saddle points	Gradient ≈ 0 in flat regions/saddle points; GD can get stuck or move extremely slowly.	Momentum, NAG, Adam
Oscillation in ravines	In narrow, steep valleys GD zig-zags across the walls instead of moving along the valley floor.	Momentum, NAG
Vanishing adaptive rates	AdaGrad's ever-growing denominator can shrink the learning rate to near zero, halting learning.	RMSprop, Adam (decaying average)
Noisy gradients (mini-batch/SGD)	Small or single-sample batches give noisy, unstable gradient estimates.	Momentum-style averaging, larger batches

In short: the central tension every optimizer is trying to resolve is **speed vs. stability** — moving fast enough to converge in reasonable time, without overshooting, oscillating, or getting trapped in poor regions of the loss surface.

5. What Next? — How to Choose an Optimizer

Practical guidance for picking an optimizer in real projects:

- **Start with Adam** for most deep learning tasks — it's robust, requires little tuning, and converges reliably (default: $\text{lr} = 0.001$).
- **Use SGD with Momentum** when you want potentially better final generalization and have time to tune the learning rate and momentum carefully — common in some computer vision research settings.
- **Use RMSprop** for recurrent neural networks and other non-stationary problems where it has historically performed well.
- **Watch the learning rate above all else** — even the best optimizer fails with a badly chosen learning rate. Consider learning-rate schedules or warmup strategies.
- **Monitor the loss curve** — oscillating loss suggests the learning rate is too high; an extremely slow decrease suggests it's too low or the optimizer is stuck.

Going further, the natural next topics to study are: **learning rate schedulers** (step decay, cosine annealing, warmup), **AdamW** (Adam with corrected weight decay, widely used in transformer training), and **second-order methods** (e.g. Newton's method, L-BFGS) which use curvature information but are rarely practical for large neural networks.

Note: These notes summarize standard, well-established deep learning concepts related to this topic (Gradient Descent, Momentum, AdaGrad, RMSprop, Adam) for self-study. They are not a transcript of any specific video.